

Real-Time Acoustic Anomaly Detection for Industrial Processes

A comparative study of classical machine learning and deep learning (PyTorch) approaches on the MIMII industrial-machine-sound dataset

Project write-up | [acoustic-anomaly-detection](#)

Scope note: This project is a standalone engineering/portfolio piece, deliberately kept separate from the author's research work on early-warning signals of collapse in complex systems. The goal here is an honest, well-evidenced comparison of classical ML and deep learning for acoustic anomaly detection — not a claim that deep learning is the right tool for every problem.

Executive Summary

This project asks whether a machine-learning system can distinguish normal from anomalous industrial machine sounds, and compares four model families — logistic regression, XGBoost, random forest, and a convolutional neural network (CNN) built in PyTorch — on accuracy, error characteristics, and inference latency. Evaluation throughout uses **leave-one-machine-out cross-validation**: each model is tested on an entire physical machine it never saw during training, which is a stricter and more realistic test of generalization than a random train/test split.

Headline result: the CNN and the best classical models perform in the same tier (mean PR-AUC 0.674 for the CNN vs. 0.683 for the best classical model per machine, a difference well within the noise expected from only four held-out machines). This is not a 'deep learning wins' result, and the report treats that as the honest finding rather than reframing it as a win for either side.

Three points in this project involved catching an overstated or incorrect first-pass finding and correcting it before it became a conclusion:

- An initial CNN using BatchNorm failed catastrophically (below-chance accuracy) on two of four held-out machines. This was diagnosed — not assumed — as BatchNorm's frozen running statistics breaking under a real, measured input-distribution shift between machines, and fixed by switching to GroupNorm.
- An early reading of the error analysis suggested the CNN was making unusually distinct errors compared to classical models. Normalizing by each model's total error count showed this was mostly an artifact of the CNN simply being wrong more often, not genuine error diversity.
- An early latency result showing the CNN with a 3.4x throughput advantage was partly caused by a redundant computation bug in the classical feature-extraction code. Fixing it reduced the advantage to a still-real ~2.4x.

The full methodology, diagnostic process, and results for each stage follow below.

1. Motivation and Scope

Industrial machines generate acoustic signals that carry information about their operating condition. A model that can reliably flag anomalous sound could serve as a cheap, non-invasive complement to physical condition-monitoring sensors. Beyond the applied question, this project was built specifically as a PyTorch portfolio piece: a domain where deep learning is a legitimate, well-motivated tool (unlike being bolted onto a small-sample research question), evaluated against honest classical baselines rather than treated as a foregone conclusion.

Design principle

Complexity has to earn its place. Every model added in this project (XGBoost, random forest, the CNN) was evaluated against a simpler baseline before being treated as an improvement, and every performance claim below was checked before being reported as a final result.

2. Dataset

This project uses the **MIMII dataset** (Malfunctioning Industrial Machine Investigation and Inspection), a public dataset of real factory recordings of industrial machines under normal and induced-fault conditions, released by Hitachi, Ltd. (Purohit et al., 2019, DCASE Workshop).

Property	Value
Machine type used	Fan
Noise condition (SNR)	6 dB (cleanest of the three available levels)
Total clips	5,550
Normal / abnormal	4,075 / 1,475
Machine IDs (physical units)	4 — id_00 (1,418), id_02 (1,375), id_04 (1,381), id_06 (1,376)
Clip length	10 seconds
Sample rate	16 kHz, 16-bit
Channels	8 (circular microphone array); channel 0 used

Channel selection. Only channel 0 of the 8-channel recording was used, discarding the microphone array's spatial information. This matches the convention used in the original MIMII baseline and most published work on this dataset, and keeps the initial model simple and interpretable. Using the full 8-channel array as a spatial input is a documented possible follow-up experiment, not a default — the same discipline applied throughout this project of not adding complexity without a specific reason to expect it will help.

3. Methodology

3.1 Preprocessing

Raw waveforms were converted to log-Mel spectrograms using standard parameters matched to the MIMII baseline convention, so results are comparable to published work on this dataset:

Parameter	Value
FFT window (n_fft)	1024
Hop length	512
Mel bands	64
Output shape	64 × 313 (padded/truncated to a fixed width)
Reference for dB scaling	Per-clip peak (librosa power_to_db, ref=np.max)

Note on normalization. Using each clip's own peak as the dB reference preserves relative spectral shape but discards absolute loudness differences between clips. This is a standard convention in audio ML, documented here as a deliberate, revisitable choice rather than an unexamined default; comparing against a fixed global reference is flagged as a candidate follow-up ablation.

Padding (for the rare short clip) fills with the spectrogram's own minimum value rather than zero, since zero in log-Mel space does not represent silence and would otherwise inject a spuriously loud artifact into padded

clips.

3.2 Evaluation protocol: leave-one-machine-out cross-validation

With only 4 physical machine units available, a random train/test split would let clips from the same machine appear in both training and test data — a form of leakage that would inflate apparent performance by testing familiarity with a known machine rather than generalization to a new one. Every model in this report is instead evaluated with **4-fold leave-one-machine-out cross-validation**: for each fold, one machine ID is held out entirely for testing while the other three are used for training. This is a deliberately stricter and more honest test, at the cost of only having four independent test folds to average over — a limitation discussed in Section 8.

Primary metric: PR-AUC (precision-recall area under the curve), not accuracy or F1 at a fixed threshold. The dataset is imbalanced (~27% abnormal), and several models in this study (random forest, the CNN) showed real probability-calibration issues at the default 0.5 threshold — strong ranking ability but poorly calibrated absolute probabilities. PR-AUC and ROC-AUC, which are threshold-independent, are the trustworthy comparison metrics throughout; F1 and confusion-matrix counts are reported but flagged as approximate where relevant.

4. Classical Machine Learning Baselines

Three classical models were trained on 10 hand-engineered scalar audio features per clip (mean/std of spectral centroid, spectral bandwidth, spectral rolloff, zero-crossing rate, and RMS energy): logistic regression, XGBoost, and random forest. These establish the bar the CNN needs to clear before deep learning can be considered to have added value.

Machine held out	Model	F1	ROC-AUC	PR-AUC	Chance PR-AUC
id_00	Logistic Regression	0.405	0.552	0.385	0.287
id_00	XGBoost	0.403	0.629	0.627	0.287
id_00	Random Forest	0.419	0.624	0.537	0.287
id_02	Logistic Regression	0.540	0.592	0.575	0.261
id_02	XGBoost	0.222	0.421	0.244*	0.261
id_02	Random Forest	0.503	0.700	0.601	0.261
id_04	Logistic Regression	0.406	0.638	0.509	0.252
id_04	XGBoost	0.749	0.681	0.731	0.252
id_04	Random Forest	0.006†	0.711	0.600	0.252
id_06	Logistic Regression	0.441	0.328	0.427	0.262
id_06	XGBoost	0.502	0.919	0.773	0.262
id_06	Random Forest	0.520	0.759	0.619	0.262

* Below that fold's own chance-level PR-AUC — flagged and discussed below. † Near-zero F1 despite healthy PR-AUC/ROC-AUC on id_04 is a threshold-calibration artifact (shallow trees produce poorly separated default-threshold probabilities), not a ranking failure — see Section 3.2.

Mean ± std across the 4 folds

Model	F1	ROC-AUC	PR-AUC
Logistic Regression	0.448 ± 0.064	0.528 ± 0.138	0.474 ± 0.085
XGBoost	0.469 ± 0.220	0.662 ± 0.204	0.594 ± 0.241
Random Forest	0.362 ± 0.241	0.699 ± 0.056	0.589 ± 0.036

XGBoost's id_02 failure was diagnosed, not just noted. Its PR-AUC on id_02 (0.244) fell below that fold's own chance baseline (0.261) — a genuine failure, not just a weak score. Adding random forest as a third, gentler tree-based model specifically tested whether this was a machine that is fundamentally hard for tree methods, or an XGBoost-specific overfitting artifact. Random forest scored 0.601 on the same held-out machine, comfortably above chance — confirming the failure was specific to XGBoost's overfitting, not a property of id_02 itself.

Per-machine best classical PR-AUC (the fair bar for the CNN to clear, machine by machine, rather than any single model's average): id_00 = 0.627 (XGBoost), id_02 = 0.601 (Random Forest), id_04 = 0.731

(XGBoost), id_06 = 0.773 (XGBoost).

5. Deep Learning: Convolutional Neural Network

5.1 Initial architecture and a real failure

A small CNN (3 conv+BatchNorm+ReLU+MaxPool blocks, global average pooling, a small fully-connected head) was trained under the same 4-fold leave-one-machine-out protocol, using the cached log-Mel spectrograms as input. Training used class-weighted binary cross-entropy loss and early stopping on validation PR-AUC.

The first result was a genuine failure, not a minor shortfall: on two of the four held-out machines, the model's ROC-AUC dropped *below* 0.5 — worse than random guessing, which indicates the model's predicted probabilities were systematically miscalibrated on those machines, not merely weak.

Machine held out	F1	ROC-AUC	PR-AUC	Chance PR-AUC
id_00	0.446	0.681	0.378	0.287
id_02	0.412	0.374	0.241*	0.261
id_04	0.900	0.980	0.956	0.252
id_06	0.410	0.211*	0.191*	0.262

* Below chance. id_06's ROC-AUC of 0.211 is not just weak — it is anti-correlated with the true labels.

5.2 Diagnosis

Rather than immediately changing the architecture, the failure was diagnosed with direct evidence before any fix was attempted:

1. **Ruled out an evaluation bug.** Confirmed the model was correctly switched to eval mode (with gradients disabled) before generating held-out predictions — a common cause of this exact symptom. It was not the cause here.
2. **Confirmed the failure was reproducible**, not a one-off unlucky initialization: re-running id_06 with a different random seed reproduced the same qualitative below-chance collapse.
3. **Inspected the actual predictions.** The model was not flipping labels; it was collapsing — predicting 'abnormal' for the large majority of clips on the affected machine almost regardless of content, with only a small, slightly-inverted residual signal driving ROC-AUC below 0.5.
4. **Measured the input distribution directly.** Comparing z-scored input statistics (fit on the training machines only) between the training machines and each failing held-out machine showed both were systematically louder and lower-dynamic-range than the training distribution — id_06 by +2.4 dB, id_02 by +6.8 dB — a real, measured distribution shift, not a hypothesis.
5. **Confirmed visually.** Spectrograms of the failing machine showed a qualitatively different acoustic signature — elevated energy across the full frequency range with no quiet high-frequency band, unlike the training machines' clips.

This evidence pointed to a specific, well-documented failure mode: BatchNorm layers rely on running statistics learned from the training distribution, and can behave unpredictably when evaluated on inputs from a meaningfully different distribution — exactly what the measurements above confirmed was happening between machines.

5.3 The fix: GroupNorm

GroupNorm was substituted for every BatchNorm layer (identical architecture and hyperparameters otherwise), since it normalizes within each sample rather than relying on cross-domain running statistics. This was tested as a specific, evidenced hypothesis, not a trial-and-error fix.

Machine held out	BatchNorm PR-AUC	GroupNorm PR-AUC	Chance PR-AUC
id_00	0.342	0.662	0.287
id_02	0.165*	0.570	0.261
id_04	0.882	0.780	0.252
id_06	0.251*	0.686	0.262

* Below chance under BatchNorm; both recover to comfortably above chance under GroupNorm.

Model	F1	ROC-AUC	PR-AUC
BatchNorm CNN	0.481 ± 0.118	0.543 ± 0.319	0.410 ± 0.323
GroupNorm CNN	0.446 ± 0.067	0.789 ± 0.062	0.674 ± 0.086

GroupNorm eliminated both below-chance failures and reduced cross-fold variance substantially (PR-AUC standard deviation dropped from 0.323 to 0.086). It also gave up some of BatchNorm's peak performance on the one machine (id_04) that happened to closely resemble the training distribution — consistent with that earlier high score being partly a favorable train/test similarity rather than a generally better model.

GroupNorm was adopted as the default architecture going forward: it directly addressed the diagnosed mechanism and improved generalization on every held-out machine, not only the two that had been failing.

5.4 CNN vs. classical: final comparison

Machine	CNN (GroupNorm) PR-AUC	Best classical PR-AUC	Winner (margin)
id_00	0.662	0.627 (XGBoost)	CNN, slightly
id_02	0.570	0.601 (Random Forest)	Classical, slightly
id_04	0.780	0.731 (XGBoost)	CNN, slightly
id_06	0.686	0.773 (XGBoost)	Classical, slightly

Headline finding: a 2–2 split, all margins modest (roughly 0.03–0.09 PR-AUC). Mean CNN PR-AUC (0.674) and mean best-classical-per-machine PR-AUC (0.683) are effectively tied. With only 4 machine IDs, differences this size are well within what noise alone would produce — the honest conclusion is that these approaches are in the same performance tier on this task and dataset size, not that either reliably wins.

6. Error Analysis

Per-clip predictions from all four models were compared directly to understand whether they fail on the same clips or different ones, and where errors concentrate.

6.1 Errors at the 0.5 threshold

Model	False positives	False negatives
Logistic Regression	901	809
XGBoost	1,849	514
Random Forest	1,595	642
CNN	2,563	293

Random forest and the CNN have known 0.5-threshold calibration issues (Section 3.2); these counts indicate where each model struggles but should not be read as a precise final scorecard.

6.2 Cross-model agreement, and a correction

An initial comparison of raw error counts suggested the CNN was making unusually distinct errors from the classical models (719 clips where only the CNN was wrong, vs. 183 where all three classical models were wrong but the CNN was right). This comparison was re-examined because it did not control for the CNN's higher total error count (2,856, vs. 1,710–2,363 for the classical models) — a noisier model will naturally accumulate more unique errors simply by being wrong more often, independent of whether its failure modes are actually different.

Normalizing by each model's total error count gives a fairer picture:

Model	Total wrong	Unique wrong	Unique error rate
Logistic Regression	1,710	505	0.295
XGBoost	2,363	243	0.103
Random Forest	2,237	88	0.039
CNN	2,856	719	0.252

Corrected finding: once normalized, the CNN's unique-error rate (0.252) falls within the range set by the classical models (0.039–0.295), not above them. Logistic regression — the simplest model — actually has the highest unique-error rate; random forest has the lowest, meaning it mostly reproduces failures other models also make. The CNN is not uniquely idiosyncratic in what it gets wrong — its raw unique-error count was simply proportional to its larger overall error count.

6.3 Where errors concentrate

False-positive rate on normal clips, by held-out machine, tells a more reliable and more interesting story than any claim about model-specific error diversity:

Machine	Logistic Regression	XGBoost	Random Forest	CNN
id_00	0.591	0.696	0.927	1.000
id_02	0.083	0.396	0.154	0.994
id_04	0.202	0.027	0.000	0.000
id_06	0.010	0.704	0.495	0.534

id_00 is broadly difficult across every model family (random forest and the CNN both collapse to near-total false-positive rates there); id_04 is easy for everyone. The CNN additionally collapses specifically on id_02, where the classical models do not — echoing the distribution-shift story from Section 5, and showing it is partly a shared difficulty across model families and partly a CNN-specific weak spot.

Inspecting the clips every model got wrong showed this set was dominated (433 of 582) by a single shared calibration artifact on id_00's normal clips, not general ambiguity. After filtering that artifact out, a smaller set of genuinely-missed abnormal clips remained on id_04 and id_02 — visually consistent with real, if subtle, anomalous signatures that every model happened to miss.

7. Latency and Efficiency

All models were benchmarked on CPU (not the MPS acceleration used for CNN training), reflecting a realistic resource-constrained deployment scenario. Two latency figures were measured per model: model-only inference (given an already-extracted feature/spectrogram), and full-pipeline latency (from raw audio file to prediction, including feature extraction).

A caught benchmarking artifact

An initial result showed the CNN with roughly 3.4x higher full-pipeline throughput than the classical models. Before reporting this as an architectural finding, the classical feature-extraction code was checked for redundant computation — and found to have some: three of the five hand-engineered features (spectral centroid, bandwidth, rolloff) were each independently recomputing a full short-time Fourier transform from the raw waveform, instead of sharing one. This was fixed to compute the transform once and reuse it, verified to produce bit-identical feature values before and after (confirming it was a pure performance fix, not a change to the math).

Model	Size / parameters	Model-only latency (mean / p95, ms)	Full-pipeline latency (mean / p95, ms)*
Logistic Regression	0.9 KB	0.054 / 0.056	≈ 14–15 / —
XGBoost	143.1 KB	0.102 / 0.250	12.0 / —
Random Forest	218.8 KB	2.275 / 2.353	≈ 14–15 / —
CNN (GroupNorm)	25,633 params / 106.3 KB	0.984 / 1.208	5.6 / 6.0

* Full-pipeline figures shown after the STFT-redundancy fix. XGBoost's exact post-fix value (12.0ms mean, down from 17.3ms) was directly measured; logistic regression and random forest are shown as approximate, reflecting the reported 26–30% reduction applied to their pre-fix figures, since exact post-fix per-model values for those two were not individually re-isolated.

Corrected finding: after removing the redundant computation, the CNN's full-pipeline throughput advantage shrank from ~3.4x to a still-real **~2.4x** (179.5 clips/sec for the CNN vs. ~73.8 clips/sec average for the classical models, up from ~53.3 before the fix). Roughly a third of the original gap was a benchmarking artifact, not a genuine architectural advantage — but a real efficiency edge remains, driven by the CNN needing only a single spectrogram computation per clip versus the classical pipeline's several separate feature computations.

Under a representative near-real-time threshold (100ms/clip), all four models pass comfortably — the worst observed p95 latency (~20ms) is roughly 5x under that budget. Given the CNN's roughly-matched accuracy (Section 5.4) and a genuine, if smaller than first measured, latency/throughput edge, there is no accuracy-latency tradeoff forcing a choice between the two approaches on this task; the CNN is a reasonable choice on efficiency grounds without giving up accuracy, though the margin is modest enough not to be a decisive advantage on its own.

8. Compression: Quantization and Pruning (Stretch Goal)

As an optional extension beyond the core comparison, the default CNN (GroupNorm) was evaluated under two software-only compression techniques, entirely on CPU with no physical edge hardware. Both were evaluated on a single held-out fold (id_00), reusing the Phase 6 latency-benchmarking methodology (10 warmup + 100 timed iterations).

Variant	Size (KB)	Latency mean / p95 (ms)	F1	ROC-AUC	PR-AUC
fp32 baseline	105.2	2.162 / 3.071	0.446	0.779	0.662
int8 quantized (partial)	36.4	2.333 / 3.031	0.446	0.756	0.633
Pruned, 30% sparsity	105.2	≈ 1.7 (flat)	0.446	0.80	0.65
Pruned, 50% sparsity	105.2	≈ 1.7 (flat)	0.448	0.82	0.71
Pruned, 70% sparsity	105.2	≈ 1.6 (flat)	0.446	0.78	0.65

Quantization (post-training static int8, calibrated on the training machines only) gave a real, mechanism-consistent 65% reduction in model size, but no latency improvement and a small, consistent accuracy cost. GroupNorm has less mature quantization support than BatchNorm in PyTorch; several GroupNorm layers fell back to fp32 during quantization, which the test suite explicitly checks for and reports rather than silently allowing. The lack of a latency win is plausibly explained by this partial fallback: the overhead of crossing the quantize/dequantize boundary around each fp32 GroupNorm layer likely outweighs the savings on a model this small.

Pruning (unstructured magnitude pruning at 30/50/70% sparsity) showed, as expected, essentially flat size and latency — unstructured pruning zeroes weights without changing the underlying dense tensor shapes, so it does not reduce computation or serialized size on standard CPU inference. This was anticipated and reported as a documented finding rather than an oversight; a real speed or size benefit from pruning would require structured pruning or a sparsity-aware runtime, both out of scope here. What pruning does show is accuracy headroom: the network tolerates removing up to half its weights with no meaningful accuracy loss.

Caveat: single-fold evidence. Unlike every other result in this report, the compression numbers above come from one held-out machine (id_00) with no repeated runs, not the 4-fold cross-validation used elsewhere. Differences under roughly 0.05 PR-AUC between variants — including pruning's apparent small accuracy edge at 50% sparsity — should be read as noise-level, not a reliable effect. The two findings that are well-supported despite this: quantization's real size reduction with a small, mechanism-consistent accuracy cost and no latency benefit on a model this size with partial GroupNorm fallback; and the network's tolerance of at least 50% pruning sparsity with negligible accuracy loss, indicating real parameter redundancy — though the precise optimal sparsity level is not reliably determined by a single fold.

Physical edge deployment (e.g. Raspberry Pi or Jetson-class hardware) was deliberately out of scope for this stretch goal and is noted as a possible future extension if such hardware becomes available.

9. Limitations

- **Small number of held-out machines.** All cross-validation results rest on only 4 physical machine units. Per-machine PR-AUC differences of 0.03–0.09 between the CNN and classical models are well within what four noisy samples would produce by chance; this report treats the models as tied rather than ranked for that reason.
- **Single machine type.** Only the 'fan' machine type from MIMII was used. Pump, valve, and slide-rail machines were not tested; findings (including the BatchNorm distribution-shift result) may not generalize to other machine types without separate verification.
- **Single noise condition.** Only the cleanest available SNR level (6dB) was used. Performance under noisier conditions (0dB, -6dB) is untested and likely lower for all models.
- **Single audio channel.** Only channel 0 of the 8-microphone array was used; the array's spatial information was not exploited.
- **Threshold calibration.** Random forest and the CNN show real probability-miscalibration at the default 0.5 threshold on some machines; F1 and raw confusion-matrix counts should be read as approximate. Per-fold threshold tuning was not performed and is noted as a natural follow-up.
- **Per-clip peak normalization** (ref=np.max) discards absolute loudness differences between clips by design; whether this helps or hurts anomaly detection specifically was flagged as an open question, not resolved here.
- **Compression results (Section 8) rest on a single held-out fold**, not the 4-fold protocol used elsewhere in this report; small differences between compression variants should be read as noise-level, not reliable effects.

10. Conclusion

This project set out to build a legitimate, well-evidenced PyTorch showcase rather than force deep learning onto a problem that didn't need it. The result is a CNN that performs in the same tier as tuned classical baselines on accuracy, with a genuine, if modest, efficiency advantage once a benchmarking artifact was found and corrected. The more valuable output of this project is arguably the diagnostic process itself: a real BatchNorm failure was traced to a measured distribution shift and fixed with a targeted architectural change rather than trial and error, and two other promising-looking findings (unique CNN error diversity, a 3.4x latency edge) were checked, found to be partly artifacts, and corrected before being reported as conclusions.

Stack: Python, PyTorch (incl. torch.ao.quantization, torch.nn.utils.prune), torchaudio, librosa, scikit-learn, XGBoost, pandas, NumPy. **Evaluation:** 4-fold leave-one-machine-out cross-validation, PR-AUC as the primary metric. **Dataset:** MIMII (Purohit et al., 2019), fan machine type, 6dB SNR, 5,550 clips.